

리눅스 스토리지 관리

LFCS 도메인 5 (배점 20%) — 파티션, 파일시스템, 마운트, LVM, RAID, 스왑, NFS, 성능 모니터링

목차

- 1. 블록 디바이스 이해하기
- 2. 파티션 관리
- 3. 파일시스템 생성
- 4. 마운트 — 수동과 자동(fstab)
- 5. 파일시스템 옵션 조정
- 6. LVM — 논리 볼륨 관리
- 7. RAID
- 8. 스왑 공간
- 9. 디스크 사용량 관리
- 10. NFS — 원격 파일시스템
- 11. 원격 블록 디바이스 (iSCSI)
- 12. 스토리지 성능 모니터링
- 13. 암호화 스토리지 (LUKS)
- 14. 실습 체크리스트 / 자주 하는 실수

1. 블록 디바이스 이해하기

1.1 계층 구조

스토리지는 아래에서 위로 쌓입니다. 이 순서를 머리에 넣으면 모든 명령의 위치가 정리됩니다.

```
물리 디스크      /dev/sdb
  ↓ (파티션)
파티션          /dev/sdb1
  ↓ (선택: LVM 또는 RAID)
논리 볼륨       /dev/vg_data/lv_app
  ↓ (파일시스템 생성)
파일시스템      ext4 / xfs
  ↓ (마운트)
디렉터리        /mnt/data
```

1.2 장치 이름 규칙

이름	의미
/dev/sda , sdb	SATA/SAS/USB 디스크. 순서대로 a, b, c
/dev/sda1 , sda2	해당 디스크의 파티션
/dev/nvme0n1	NVMe SSD (0번 컨트롤러, 1번 네임스페이스)
/dev/nvme0n1p1	NVMe의 파티션. p 가 볼륨
/dev/vda	virtio 가상 디스크 (KVM/클라우드)
/dev/mapper/vg-lv	LVM 논리 볼륨

1.3 현황 확인

```
lsblk                # 트리 형태로 한눈에 (가장 자주 씀)
lsblk -f             # 파일시스템 타입과 UUID까지
sudo fdisk -l        # 파티션 테이블 상세
sudo blkid           # UUID와 타입
df -h               # 마운트된 파일시스템 사용량
sudo parted -l       # 파티션 테이블 종류 포함
```

새 디스크를 붙였을 때는 **lsblk -f** 가 첫 명령입니다. 장치 이름, 크기, 파일시스템 유무, 마운트 지점을 한 번에 보여줍니다. 출력에서 FSTYPE이 비어 있으면 아직 포맷되지 않은 디스크입니다.

작업 전 반드시 대상 장치를 재확인하세요. /dev/sda 는 보통 부팅 디스크입니다. 새로 붙인 디스크라고 생각하고 sdb 대신 sda 를 포맷하면 시스템이 날아갑니다. lsblk 로 크기와 마운트 상태를 대조해 확실히 구분한 뒤 진행하세요.

2. 파티션 관리

2.1 MBR과 GPT

	MBR	GPT
최대 디스크	2TB	사실상 무제한 (9.4ZB)
파티션 수	주 4개 (확장으로 우회)	128개
부팅	BIOS	UEFI
권장	레거시 호환용	신규 구성은 GPT

2.2 fdisk (대화형)

```
sudo fdisk /dev/sdb
```

키	동작	키	동작
m	도움말	n	새 파티션
p	현재 테이블 출력	d	파티션 삭제
t	타입 변경	g	새 GPT 테이블
w	저장 후 종료	q	저장 없이 종료

w 를 누르기 전까지는 아무것도 디스크에 기록되지 않습니다. 실수했다면 q 로 빠져나오면 원래대로입니다. 이 안전장치 덕분에 p 로 결과를 충분히 확인한 뒤 w 를 누르는 습관을 들일 수 있습니다.

2.3 parted (비대화형 가능)

```
sudo parted /dev/sdb mklabel gpt
sudo parted -a optimal /dev/sdb mkpart primary ext4 0% 100%
sudo parted /dev/sdb print
```

-a optimal 은 파티션 시작 위치를 물리적 경계에 정렬합니다. SSD와 RAID에서 정렬이 어긋나면 성능이 크게 떨어지므로 붙이는 것이 좋습니다.

2.4 커널에 변경 알리기

```
sudo partprobe /dev/sdb
sudo partx -u /dev/sdb
lsblk
```

파티션을 만들었는데 /dev/sdb1 이 안 보이면 커널이 아직 테이블을 다시 읽지 않은 것입니다. 사용 중인 디스크는 재부팅이 필요할 수도 있습니다.

3. 파일시스템 생성

3.1 종류 선택

종류	특징	축소	주 용도
ext4	안정적, 범용. 오래 검증됨	가능	일반 서버, 기본 선택
XFS	대용량·병렬 I/O 강함. RHEL 기본	불가	대용량 데이터, DB
Btrfs	스냅샷, 압축 내장	가능	스냅샷 필요 환경
vfat	범용 호환	-	EFI 부트 파티션, USB

XFS는 축소가 불가능합니다. 한 번 키운 XFS 파일시스템은 줄일 수 없습니다. 나중에 용량 조정 가능성이 있다면 ext4를 선택하거나, LVM으로 여유를 두고 필요할 때 확장하는 전략을 쓰세요. RHEL 계열의 기본이 XFS라는 점을 모르고 나중에 낭패를 보는 경우가 많습니다.

3.2 생성

```
sudo mkfs.ext4 /dev/sdb1
sudo mkfs.xfs /dev/sdb1
sudo mkfs.ext4 -L data /dev/sdb1    # 레이블 지정
sudo mkfs.xfs -f /dev/sdb1         # 기존 파일시스템 강제 덮어쓰기
```

3.3 검사와 레이블

```
sudo fsck /dev/sdb1                # ext 계열 검사 — 반드시 언마운트 상태에서
sudo fsck -y /dev/sdb1             # 자동 수정
sudo xfs_repair /dev/sdb1          # XFS 전용

sudo e2label /dev/sdb1 data        # ext 레이블 변경
sudo xfs_admin -L data /dev/sdb1   # XFS 레이블 변경
sudo blkid /dev/sdb1               # UUID 확인
```

마운트된 상태에서 **fsck** 를 실행하면 파일시스템이 손상됩니다. 반드시 언마운트하고 실행하세요. 루트 파일시스템은 언마운트할 수 없으므로 복구 모드로 부팅하거나 `touch /forcefsck` 후 재부팅합니다.

4. 마운트 — 수동과 자동(fstab)

4.1 수동 마운트

```
sudo mkdir -p /mnt/data
sudo mount /dev/sdb1 /mnt/data
sudo mount -t xfs /dev/sdb1 /mnt/data    # 타일 명시
sudo mount -o ro /dev/sdb1 /mnt/data     # 읽기 전용

mount | column -t                    # 현재 마운트 목록
findmnt                             # 트리 형태 (더 보기 좋음)
findmnt /mnt/data                    # 특정 지점

sudo umount /mnt/data
sudo umount -l /mnt/data              # lazy — 사용 중이어도 분리 예약
```

언마운트가 안 될 때

```
sudo lsof +D /mnt/data      # 해당 경로를 쓰는 프로세스
sudo fuser -vm /mnt/data    # 사용 중인 프로세스 확인
sudo fuser -km /mnt/data    # 강제 종료 후 언마운트 (주의)
```

target is busy 오류의 가장 흔한 원인은 **본인의 셸이 그 디렉터리 안에 있는** 것입니다. `cd /` 로 나온 뒤 다시 시도해보세요.

4.2 fstab — 부팅 시 자동 마운트

```
sudo blkid /dev/sdb1
# /dev/sdb1: UUID="a1b2c3d4-..." TYPE="xfs"

sudo vim /etc/fstab
```

```
UUID=a1b2c3d4-... /mnt/data xfs defaults 0 2
  ①           ②           ③           ④   ⑤ ⑥
```

	필드	설명
①	장치	UUID 권장 (레이블, 장치명도 가능)
②	마운트 지점	미리 존재해야 함
③	파일시스템 타입	ext4, xfs, nfs 등
④	옵션	defaults, noatime, nofail 등
⑤	dump	백업 대상 여부. 보통 0
⑥	fsck 순서	0=검사안함, 1=루트, 2=나머지

왜 UUID를 쓰는가. 디스크를 추가하거나 순서가 바뀌면 `/dev/sdb` 가 `/dev/sdc` 로 바뀔 수 있습니다. 그러면 fstab의 장치명이 엉뚱한 디스크를 가리켜 부팅이 실패하거나 잘못된 데이터가 마운트됩니다. UUID는 파일시스템에 고정되어 있어 이 문제가 없습니다.

fstab 오류는 부팅 실패로 직결됩니다. 존재하지 않는 장치를 적으면 emergency mode로 떨어져 콘솔 접근 없이는 복구할 수 없습니다.

반드시 이 순서로 검증하세요:

```
sudo mount -a      # fstab 전체를 실제로 마운트 시도
findmnt --verify    # 문법 검증
```

오류 없이 통과해야 재부팅해도 안전합니다. 그리고 데이터 디스크에는 **nofail** 옵션을 넣으세요. 디스크가 없어도 부팅은 되게 해줍니다.

4.3 주요 마운트 옵션

옵션	효과
defaults	rw, suid, dev, exec, auto, nouser, async
ro / rw	읽기 전용 / 읽기 쓰기
noatime	읽기 시 접근 시각 갱신 안 함 — 성능 향상
relatime	절충안. 현재 기본값
noexec	실행 파일 실행 금지 — <code>/tmp</code> 보안 강화
nosuid	SUID 비트 무시
nodev	장치 파일 인식 안 함
nofail	장치가 없어도 부팅 진행

_netdev	네트워크 준비 후 마운트 (NFS/iSCSI 필수)
---------	------------------------------

```
UUID=... /tmp ext4 defaults,noexec,nosuid,nodev 0 2
UUID=... /data xfs defaults,noatime,nofail 0 2
```

5. 파일시스템 옵션 조정

5.1 재마운트로 옵션 변경

```
sudo mount -o remount,ro /mnt/data # 읽기 전용으로 전환
sudo mount -o remount,rw /         # 루트를 쓰기 가능으로 (복구 시)
```

5.2 예약 블록 조정 (ext 계열)

ext4는 기본적으로 5%를 root 전용으로 예약합니다. 시스템 파티션에서는 디스크가 가득 차도 root가 복구 작업을 할 수 있게 하는 안전장치지만, **순수 데이터 디스크에서는 낭비**입니다.

```
sudo tune2fs -m 1 /dev/sdb1 # 1%로 축소
sudo tune2fs -l /dev/sdb1   # 전체 정보 확인
```

1TB 디스크라면 5%가 50GB입니다. 데이터 전용이라면 1%로 줄여 49GB를 회수할 수 있습니다.

5.3 파일시스템 확장

```
sudo resize2fs /dev/vg_data/lv_app # ext4 — 온라인 확장 가능
sudo xfs_growfs /mnt/data          # XFS — 마운트 지점을 인자로
```

명령의 인자가 다릅니다. `resize2fs` 는 **장치**를, `xfs_growfs` 는 **마운트 지점**을 받습니다. 자주 헷갈리는 부분입니다.

6. LVM — 논리 볼륨 관리

6.1 왜 쓰는가

물리 파티션은 한 번 크기를 정하면 바꾸기 어렵습니다. LVM은 디스크와 파일시스템 사이에 추상화 계층을 넣어 다음을 가능하게 합니다.

- 여러 디스크를 합쳐 하나의 큰 볼륨으로
- 서비스 중단 없이 **온라인 확장**
- 스냅샷 생성 (백업 시점 고정)
- 디스크 추가만으로 용량 증설

6.2 3계층 구조

PV (Physical Volume)	물리 볼륨	/dev/sdb1, /dev/sdc1
↓		
VG (Volume Group)	볼륨 그룹	vg_data ← PV들을 모은 저장 풀
↓		
LV (Logical Volume)	논리 볼륨	lv_app ← 실제로 포맷·마운트하는 대상

6.3 구축 절차

```
--- 1단계: PV 생성 ---
sudo pvcreate /dev/sdb1 /dev/sdc1
sudo pvs                # 요약
sudo pvdisplay           # 상세

--- 2단계: VG 생성 ---
sudo vgcreate vg_data /dev/sdb1 /dev/sdc1
sudo vgs
```

```

sudo vgdisplay vg_data

--- 3단계: LV 생성 ---
sudo lvcreate -L 50G -n lv_app vg_data
sudo lvcreate -l 100%FREE -n lv_app vg_data    # 남은 공간 전부
sudo lvs
sudo lvdisplay

--- 4단계: 포맷과 마운트 ---
sudo mkfs.ext4 /dev/vg_data/lv_app
sudo mkdir -p /srv/app
sudo mount /dev/vg_data/lv_app /srv/app

```

-L 은 절대 크기(50G), -l 은 익스텐트 개수나 비율(100%FREE)입니다. 소문자와 대문자 차이라 헷갈리기 쉽습니다.

6.4 온라인 확장 — 실무 핵심

```

--- 디스크 추가 후 VG 확장 ---
sudo pvcreate /dev/sdd1
sudo vgextend vg_data /dev/sdd1

--- LV 확장 + 파일시스템 동시 확장 ---
sudo lvextend -L +100G -r /dev/vg_data/lv_app
sudo lvextend -l +100%FREE -r /dev/vg_data/lv_app

```

-r 옵션이 핵심입니다. (--resizefs) LV를 키운 뒤 파일시스템까지 자동으로 확장해줍니다. 이게 없으면 LV만 커지고 파일시스템은 그대로라 df -h 에 변화가 없습니다.

-r 없이 했다면 수동으로: ext4는 resize2fs /dev/vg_data/lv_app , XFS는 xfs_growfs /srv/app .

6.5 축소 (ext4만, 위험)

```

sudo umount /srv/app
sudo e2fsck -f /dev/vg_data/lv_app
sudo resize2fs /dev/vg_data/lv_app 30G    # 파일시스템 먼저
sudo lvreduce -L 30G /dev/vg_data/lv_app  # 그 다음 LV
sudo mount /dev/vg_data/lv_app /srv/app

```

축소는 순서가 반대입니다. 확장은 LV → 파일시스템 순이지만, 축소는 **파일시스템 → LV** 순입니다. 순서를 틀리면 파일시스템이 LV 밖으로 빠져나가 데이터가 파괴됩니다.

축소는 언마운트 상태에서에서만 가능하고, 실패 시 복구가 어렵습니다. **반드시 백업 후** 진행하세요. XFS는 아예 축소할 수 없습니다.

6.6 스냅샷

```

sudo lvcreate -L 10G -s -n snap_app /dev/vg_data/lv_app  # 생성
sudo mount /dev/vg_data/snap_app /mnt/snap              # 마운트해서 백업
sudo lvremove /dev/vg_data/snap_app                     # 제거

```

스냅샷은 변경분만 저장하므로 원본보다 작게 잡아도 됩니다. 다만 **할당된 크기를 초과하면 스냅샷이 무효화**되므로, 백업이 끝나면 바로 제거하는 것이 원칙입니다.

7. RAID

7.1 레벨 비교

레벨	최소 디스크	용량 효율	내결함성	특징
RAID 0	2	100%	없음	스트라이핑. 속도만. 하나 죽으면 전부 손실
RAID 1	2	50%	1개	미러링. 가장 단순하고 안전
RAID 5	3	$(n-1)/n$	1개	패리티 분산. 쓰기 성능 저하
RAID 6	4	$(n-2)/n$	2개	대용량 디스크에 권장
RAID 10	4	50%	조합에 따라	미러+스트라이프. DB에 선호

RAID는 백업이 아닙니다. RAID는 디스크 *하드웨어* 고장에 대비할 뿐입니다. 실수로 `rm -rf` 한 파일, 랜섬웨어, 논리적 손상은 모든 디스크에 그대로 복제됩니다. 별도 백업이 반드시 필요합니다.

또한 대용량 디스크(8TB 이상)에서 RAID 5는 위험합니다. 디스크 하나가 죽고 재구축하는 동안 남은 디스크에 부하가 집중되어 두 번째 고장이 날 확률이 높기 때문입니다. RAID 6이나 10을 쓰세요.

7.2 mdadm으로 구성

```
--- RAID 1 생성 ---
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb1 /dev/sdc1

--- 상태 확인 ---
cat /proc/mdstat
sudo mdadm --detail /dev/md0

--- 설정 영구화 (필수) ---
sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf
sudo update-initramfs -u          # Debian 계열

--- 포맷과 마운트 ---
sudo mkfs.ext4 /dev/md0
sudo mount /dev/md0 /mnt/raid
```

7.3 장애 대응

```
sudo mdadm --manage /dev/md0 --fail /dev/sdb1    # 고장 표시
sudo mdadm --manage /dev/md0 --remove /dev/sdb1   # 배열에서 제거
sudo mdadm --manage /dev/md0 --add /dev/sdd1      # 새 디스크 추가
watch cat /proc/mdstat                            # 재구축 진행률 확인
```

`/proc/mdstat` 에서 `[UU]` 는 두 디스크 모두 정상, `[U_]` 는 하나가 빠진 상태입니다. 재구축 중에는 진행률 바가 표시됩니다.

`mdadm.conf` 에 저장하지 않으면 **재부팅 후 배열 이름이 `/dev/md127` 같은 것으로 바뀌어** `fstab` 마운트가 실패할 수 있습니다. 구성 직후 반드시 저장하세요.

8. 스왑 공간

8.1 개념

물리 메모리가 부족할 때 디스크를 임시 메모리로 쓰는 영역입니다. 스왑이 없으면 메모리 부족 시 OOM Killer가 프로세스를 강제 종료합니다.

8.2 확인

```
free -h
swapon --show
cat /proc/swaps
```

8.3 스왑 파티션 만들기

```
sudo mkswap /dev/sdb2
sudo swapon /dev/sdb2
sudo swapoff /dev/sdb2
```

8.4 스왑 파일 만들기 (더 유연함)

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile      # 필수 — 다른 사용자가 읽으면 안 됨
sudo mkswap /swapfile
sudo swapon /swapfile
```

fstab에 등록해 영구화합니다.

```
/swapfile none swap sw 0 0
```

스왑 파일의 퍼미션이 600 이 아니면 mkswap 이 경고를 냅니다. 스왑에는 메모리 내용(비밀번호, 키 등)이 그대로 담길 수 있어 다른 사용자가 읽으면 심각한 정보 유출입니다.

8.5 swappiness 조정

```
cat /proc/sys/vm/swappiness      # 기본 60
sudo sysctl -w vm.swappiness=10  # 일시
echo "vm.swappiness=10" | sudo tee /etc/sysctl.d/99-swap.conf  # 영구
```

값이 클수록 스왑을 적극적으로 씁니다. **DB 서버는 10 이하로** 낮추는 것이 일반적입니다. 스왑에 들어간 데이터를 다시 읽는 비용이 커서 응답 시간이 급격히 나빠지기 때문입니다.

쿠버네티스는 기본적으로 **스왑이 꺼져 있어야** kubelet이 기동됩니다. 메모리 관리를 예측 가능하게 유지하기 위해서입니다. 최근 버전에서 실험적으로 지원하기 시작했지만, 여전히 비활성화가 기본 권장입니다.

9. 디스크 사용량 관리

```
df -h          # 파일시스템별 사용량
df -i          # inode 사용량 — 중요
du -sh /var/log # 디렉터리 총 크기
du -h --max-depth=1 /var | sort -rh | head -10 # 용량 큰 하위 디렉터리 순위
ncdu /var      # 대화형 탐색 (설치 필요, 매우 편리)
```

"디스크가 가득 찼는데 du로는 용량이 안 보인다"는 상황의 원인은 셋 중 하나입니다.

- ① **inode 고갈** — `df -i`를 확인하세요. 작은 파일이 수백만 개면 용량이 남아도 파일을 못 만듭니다.
- ② **삭제됐지만 열려 있는 파일** — 프로세스가 붙잡고 있으면 `rm` 해도 공간이 반환되지 않습니다. `sudo lsof +L1`로 찾아 해당 프로세스를 재시작하세요. 로 그 파일을 지웠는데 용량이 안 줄어드는 전형적 사례입니다.
- ③ **마운트에 가려진 파일** — 마운트 지점 아래에 원래 있던 파일은 보이지 않지만 공간은 차지합니다.

디스크 쿼터

```
sudo mount -o remount,usrquota,grpquota /home
sudo quotacheck -cug /home
sudo quotaon /home
sudo edquota -u jihoon # 사용자별 한도 설정
sudo repquota /home   # 현황 보고
```

10. NFS — 원격 파일시스템

10.1 서버 설정

```
sudo apt install nfs-kernel-server # Debian 계열
sudo dnf install nfs-utils         # RHEL 계열

sudo mkdir -p /srv/share
sudo chown nobody:nogroup /srv/share

sudo vim /etc/exports
```

```
/srv/share 192.168.10.0/24(rw,sync,no_subtree_check)
/srv/ro    192.168.10.0/24(ro,sync,no_subtree_check)
```

export 옵션

옵션	의미
<code>rw / ro</code>	읽기 쓰기 / 읽기 전용
<code>sync</code>	디스크에 실제로 쓴 뒤 응답. 안전하지만 느림
<code>async</code>	메모리에만 쓰고 응답. 빠르지만 정전 시 데이터 손실
<code>root_squash</code>	클라이언트 root를 nobody로 강등. 기본값, 유지 권장
<code>no_root_squash</code>	클라이언트 root를 서버 root로 인정. 매우 위험
<code>no_subtree_check</code>	하위 트리 검사 생략. 성능 향상, 권장

`no_root_squash`는 클라이언트에서 root가 된 사람이 **서버 파일을 root 권한으로 조작**할 수 있게 만듭니다. 사실상 서버를 내주는 설정입니다. 특별한 이유

가 없다면 절대 쓰지 마세요.

```
sudo exportfs -ra          # 설정 재적용
sudo exportfs -v           # 현재 export 목록
sudo systemctl enable --now nfs-server
```

10.2 클라이언트

```
sudo apt install nfs-common

showmount -e 192.168.10.20      # 서버의 공유 목록 조회
sudo mount -t nfs 192.168.10.20:/srv/share /mnt/nfs
```

fstab 등록:

```
192.168.10.20:/srv/share /mnt/nfs nfs defaults,_netdev,nofail 0 0
```

_netdev 가 없으면 부팅이 멈출 수 있습니다. 네트워크가 올라오기 전에 마운트를 시도하다 타임아웃을 기다리며 부팅이 수 분간 정지합니다. **nofail** 과 함께 반드시 넣으세요.

NFS 서버가 죽으면 클라이언트의 프로세스가 **D 상태(uninterruptible sleep)**로 멈춰 **kill -9** 로도 죽지 않습니다. 중요한 환경에서는 **soft,timeo=** 옵션으로 타임아웃을 두는 것을 검토하세요.

10.3 UID 일치 문제

NFS는 **UID/GID 숫자로 권한을 판단**합니다. 서버의 jihoon이 UID 1001인데 클라이언트의 jihoon이 1005라면, 클라이언트에서 접근할 때 권한이 맞지 않습니다. 해결책은 두 가지입니다.

- 모든 서버에서 UID/GID를 동일하게 맞춤 (소규모)
- LDAP/SSSD 등 중앙 인증 도입 (권장)

11. 원격 블록 디바이스 (iSCSI)

NFS가 *파일* 단위 공유라면, iSCSI는 *블록 디바이스* 자체를 네트워크로 제공합니다. 클라이언트는 로컬 디스크처럼 인식해 직접 파티션·포맷할 수 있습니다.

	NFS	iSCSI
공유 단위	파일시스템	블록 디바이스
동시 접근	여러 클라이언트 가능	기본적으로 1:1 (클러스터FS 필요)
포맷 주체	서버	클라이언트
용도	파일 공유	DB 스토리지, VM 디스크

이니시에이터(클라이언트) 측

```
sudo apt install open-iscsi

# 타겟 검색
sudo iscsiadm -m discovery -t st -p 192.168.10.40

# 로그인 (연결)
sudo iscsiadm -m node -T iqn.2026-01.com.example:storage1 \
  -p 192.168.10.40 --login

lsblk      # 새 디스크가 /dev/sdX 로 나타남

# 이제 로컬 디스크처럼 사용
sudo mkfs.xfs /dev/sdd
sudo mount /dev/sdd /mnt/iscsi
```

fstab에는 반드시 `_netdev` 를 넣습니다.

12. 스토리지 성능 모니터링

12.1 iostat — 장치 단위

```
sudo apt install sysstat
iostat -xz 2      # 2초 간격, 확장 통계
```

지표	의미	주의 기준
%util	장치가 바쁜 시간 비율	지속 100%면 포화
await	I/O 평균 대기 시간(ms)	SSD 수 ms, HDD 수십 ms 초과 시 의심
r/s , w/s	초당 읽기/쓰기 횟수 (IOPS)	-
rkB/s , wkB/s	초당 처리량	-
aqu-sz	평균 큐 길이	1 초과면 대기 발생

NVMe나 RAID에서는 %util 이 100%여도 실제로는 여유가 있을 수 있습니다. 병렬 처리가 가능한 장치이기 때문입니다. 이런 환경에서는 await 와 aqu-sz 를 더 신뢰하세요.

12.2 pidstat — 프로세스 단위

```
pidstat -d 2          # 프로세스별 디스크 I/O
sudo iotop -o         # 실시간, I/O 중인 것만
```

iostat 과 **pidstat** 을 함께 쓰는 이유가 명확합니다. **iostat** 은 "어느 디스크가 바쁜가"를, **pidstat** / **iotop** 은 "어느 프로세스가 그 부하를 만드는가"를 알려줍니다. 장애 대응에서는 두 정보가 모두 필요합니다.

12.3 진단 순서

1. `df -h` , `df -i` — 용량과 inode가 먼저
2. `iostat -xz 2` — 어느 장치가 포화인가
3. `iotop -o` — 어느 프로세스인가
4. `lsof -p PID` — 그 프로세스가 무슨 파일을 만지는가
5. `dmesg -T | tail` — I/O 에러나 디스크 경고 확인

12.4 디스크 상태 점검

```
sudo smartctl -a /dev/sda    # SMART 정보 전체
sudo smartctl -H /dev/sda    # 건강 상태 요약
sudo smartctl -t short /dev/sda # 자체 진단 실행
```

주목할 항목: `Reallocated_Sector_Ct` , `Current_Pending_Sector` . 이 값이 0이 아니고 증가 중이면 **디스크 교체**를 준비해야 합니다.

13. 암호화 스토리지 (LUKS)

```
--- 암호화 설정 ---
sudo cryptsetup luksFormat /dev/sdb1      # 되돌릴 수 없음! 확인 후 진행

--- 열기 (매핑) ---
sudo cryptsetup luksOpen /dev/sdb1 secure_data
# /dev/mapper/secure_data 로 접근 가능해짐

--- 포맷과 마운트 ---
sudo mkfs.ext4 /dev/mapper/secure_data
sudo mount /dev/mapper/secure_data /mnt/secure

--- 닫기 ---
sudo umount /mnt/secure
sudo cryptsetup luksClose secure_data

--- 정보 확인 ---
sudo cryptsetup luksDump /dev/sdb1
```

부팅 시 자동 해제 — /etc/crypttab

```
secure_data UUID=xxxx-xxxx none luks
```

none 이면 부팅 시 비밀번호를 물어봅니다. 키 파일 경로를 지정하면 자동 해제되지만, 그 키 파일 자체의 보관이 새로운 문제가 됩니다.

LUKS 헤더가 손상되면 데이터를 영구히 복구할 수 없습니다. 구성 직후 헤더를 백업해두세요.

```
sudo cryptsetup luksHeaderBackup /dev/sdb1 \
--header-backup-file /root/luks-header.bak
```

백업 파일은 원본과 **다른 안전한 위치에** 보관해야 의미가 있습니다.

14. 실습 체크리스트 / 자주 하는 실수

14.1 실습 체크리스트

- 가상 디스크를 추가하고 `lsblk -f` 로 인식 확인
- `fdisk` 에서 `q` 로 나와 아무 변경도 없음을 확인 (안전장치 체험)
- GPT 파티션을 만들고 `partprobe` 후 장치 파일 생성 확인
- ext4와 XFS를 각각 만들고 `blkid` 로 UUID 비교
- UUID로 `fstab`을 작성하고 `mount -a + findmnt --verify` 로 검증
- 일부러 잘못된 UUID를 넣고 `mount -a` 가 오류를 내는 것 확인 (재부팅 금지)
- PV → VG → LV 순으로 LVM을 구성하고 마운트
- 디스크를 추가해 `vgextend` → `lvextend -r` 로 온라인 확장, `df -h` 로 확인
- `-r` 없이 `lvextend` 했을 때 `df -h` 가 안 변하는 것 확인
- LVM 스냅샷을 만들어 마운트한 뒤 제거
- RAID 1을 만들고 `--fail` 로 장애를 유발해 `/proc/mdstat` 의 `[U_]` 확인
- 스왑 파일을 만들고 `free -h` 로 반영 확인, `swappiness` 조정
- 큰 파일을 만든 뒤 열어놓은 채 삭제하고 `lsof +L1` 로 공간 미반환 재현
- NFS 서버/클라이언트를 구성하고 `_netdev` 유무 차이 확인

15. `iostat -xz 2` 를 띄운 채 `dd` 로 부하를 주고 지표 변화 관찰

14.2 자주 하는 실수

실수	결과	올바른 방법
대상 장치 오인 (<code>sda</code> vs <code>sdb</code>)	시스템 디스크 파괴	<code>lsblk</code> 로 크기·마운트 대조
<code>fstab</code> 에 장치명 사용	순서 바뀌면 부팅 실패	UUID 사용
<code>fstab</code> 수정 후 검증 없이 재부팅	emergency mode 진입	<code>mount -a + findmnt --verify</code>
데이터 디스크에 <code>nofail</code> 누락	디스크 부재 시 부팅 중단	<code>nofail</code> 추가
NFS/iSCSI에 <code>_netdev</code> 누락	부팅 지연 또는 실패	<code>_netdev,nofail</code>
<code>lvextend</code> 에 <code>-r</code> 누락	용량이 안 늘어난 것처럼 보임	<code>-r</code> 또는 수동 <code>resize</code>
축소 시 LV를 먼저 줄임	데이터 파괴	파일시스템 → LV 순서
XFS를 축소하려 시도	불가능	<code>ext4</code> 선택 또는 재생성
마운트 상태에서 <code>fsck</code>	파일시스템 손상	언마운트 후 실행
RAID 구성 후 <code>mdadm.conf</code> 미저장	재부팅 시 장치명 변경	<code>--detail --scan</code> 저장
RAID를 백업으로 간주	논리적 손실에 무방비	별도 백업 병행
NFS <code>no_root_squash</code>	서버 권한 탈취 위험	기본 <code>root_squash</code> 유지
스왑 파일 퍼미션 644	메모리 내용 유출	<code>chmod 600</code>
LUKS 헤더 백업 안 함	손상 시 영구 복구 불가	<code>luksHeaderBackup</code>
디스크 가득 참을 용량만 확인	inode 고갈 놓침	<code>df -i</code> 도 확인

학습용으로 새로 정리한 자료입니다. 스토리지 작업은 데이터 손실 위험이 크므로, 반드시 테스트 환경에서 충분히 연습한 뒤 백업을 확보하고 진행하시기 바랍니다.